

Atlas / Argus - Blueprint Update

FRONTEND STATUS UPDATE - 2026-05-29

This update supersedes the original PDF statements that said there was no frontend or no UI. A separate frontend archive is now present: `atlas-web-app(1).zip`. The product direction changes from "build a frontend from nothing" to "finish, harden, and wire the existing app scaffold."

What changed

- The frontend is no longer absent. The archive contains `apps/web`, a React/Vite/TypeScript/Tailwind application.
- The app includes routes for login, case list, case dashboard, claims, conflicts, timeline, provenance, reports, and audit.
- The current app is a scaffold/workbench, not a finished sellable frontend. It still needs a browser-safe BFF/session layer, real report export, document upload/ingestion screens, and a separate public marketing site.
- Build verification was not completed in this container because dependencies were not installed in the unpacked project. In the project environment, run `npm ci && npm run build` before treating the app as build-clean.

Revised product implication

Keep the original product thesis: Atlas is a defensible-record system where every projected fact is traceable to its evidence chain. The frontend update does not change the product positioning; it changes the build plan. The visible product should now be completed around the existing app shell, with conflicts, evidence, ingestion, and reports as the center of gravity.

Area	Original PDF assumption	Updated assumption
Frontend	No frontend exists.	A React/Vite app scaffold exists under <code>apps/web</code> .
App shell	Must be built from scratch.	Already started; finish auth, data wiring, layout polish, and tests.
Marketing site	Needed.	Still needed separately under <code>apps/marketing</code> .
Reports	Backend/report export gap.	Still a gap; frontend route exists but is placeholder.
Documents	PDF/docket ingestion gap.	Still a gap; no production upload/ingestion UI found.

Frontend current state found in the web app ZIP

The existing app is worth preserving. It already follows the recommended direction: React, Vite, TypeScript, Tailwind, TanStack Query, React Router, and Lucide. The route structure is close to the intended MVP.

Existing route surface

```
/login, /app, /app/cases, /app/cases/:slug,  
/app/cases/:slug/claims, /app/cases/:slug/conflicts,  
/app/cases/:slug/timeline, /app/cases/:slug/provenance,  
/app/cases/:slug/reports, /app/audit
```

Useful existing pieces

- CaseWorkspaceLayout already supports the three-column case workspace pattern recommended in the blueprint.
- TopBar, Sidebar, StatusChip, and feedback components establish a sober product UI foundation.
- lib/api.ts centralizes API access, sends cookies with credentials: include, and converts snake_case/camelCase between backend and frontend.
- The conflict screen already includes optimistic-version concepts and 409 handling hooks, but still needs safer claim-candidate data.

Updated MVP frontend priorities

Priority	Frontend work	Reason
P0	Browser-safe BFF/session auth	The browser must never hold raw API keys.
P0	Conflict queue hardening	This is the main paid workflow; fix claim candidate selection and 409 retry UX.
P0	Shared EvidencePanel	Claims, conflicts, timeline, and reports all need one consistent evidence surface.
P0	Report preview/generate/download	The report is the sellable deliverable.
P0	PDF/docket upload and ingestion status	Real users need to get evidence into the system.
P0	Marketing site and request-access form	The current ZIP has apps/web only; public lead capture is still missing.
P1	Global search activation	The top bar has a disabled search input; wire it or remove it until ready.
P1	Playwright and integration tests	Cover login, case load, conflict resolve/reopen, 409 retry, report export, and upload states.

Revised frontend roadmap for the Blueprint

- **Do not rebuild the web app from scratch.** Keep the existing apps/web scaffold and refactor only where needed.
- **Add a BFF/session boundary.** The current frontend uses cookie credentials, which is good, but the backend remains API-key based. The BFF should hold per-user API keys server-side and expose safe session endpoints.
- **Fix conflict selection.** The UI should receive full claim candidates with stable `claimId` values. Do not infer the winning claim from the index of `conflict.claimIds`.
- **Make 409 a designed state.** When a conflict is modified by another reviewer, show the returned refreshed conflict, latest activity, and current version, then let the user re-decide.
- **Unify evidence display.** Build one shared EvidencePanel used by claims, conflicts, provenance, timeline, and report preview.
- **Replace the reports placeholder.** Add preview, immutable generation, export history, and download.
- **Add document upload/ingestion UI.** Include upload, parse status, extracted text view, failures, and ingest trigger.
- **Keep the public site separate.** Add apps/marketing for Home, Platform, Use Cases, How It Works, Security, Compliance, About, Request Access, Contact, and legal pages.